

AI ASSISTANT CODING

ASSIGNMENT – 12.5

Name: P.Ashwitha

Hall Ticket Number: 2303A52420

Batch – 36

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

- Instructions:

- o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.

- o Ask AI to include time complexity and space complexity in the function docstring.

- o Verify the generated code with test cases.

```

def merge_sort(arr):
    """
    Sorts a list in ascending order using the Merge Sort algorithm.

    Time Complexity:
    - Best Case:  $O(n \log n)$ 
    - Average Case:  $O(n \log n)$ 
    - Worst Case:  $O(n \log n)$ 

    Space Complexity:
    -  $O(n)$  auxiliary space for temporary arrays used during merging.
    - Recursive stack space:  $O(\log n)$ 

    Parameters:
    arr (list): List of elements to be sorted.

    Returns:
    list: New sorted list in ascending order.
    """

    if len(arr) <= 1:
        return arr

    # Find middle index
    mid = len(arr) // 2

    # Divide the array into two halves
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    # Merge the sorted halves
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    # Compare elements from left and right arrays
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])

    return result

```

```
# ----- Test Cases -----
def test_merge_sort():
    print("Running Test Cases...\n")

    test_cases = [
        [38, 27, 43, 3, 9, 82, 10],
        [5, 2, 9, 1, 5, 6],
        [],
        [1],
        [10, 9, 8, 7, 6],
        [3, 3, 3, 3],
        [-5, -10, 0, 5, 2]
    ]

    for i, case in enumerate(test_cases, 1):
        sorted_arr = merge_sort(case)
        print(f"Test Case {i}:")
        print("Original:", case)
        print("Sorted: ", sorted_arr)
        print("-" * 30)

# Run tests
test_merge_sort()
```

... Running Test Cases...

Test Case 1:
Original: [38, 27, 43, 3, 9, 82, 10]
Sorted: [3, 9, 10, 27, 38, 43, 82]

Test Case 2:
Original: [5, 2, 9, 1, 5, 6]
Sorted: [1, 2, 5, 5, 6, 9]

Test Case 3:
Original: []
Sorted: []

Test Case 4:
Original: [1]
Sorted: [1]

Test Case 5:
Original: [10, 9, 8, 7, 6]
Sorted: [6, 7, 8, 9, 10]

Test Case 6:
Original: [3, 3, 3, 3]
Sorted: [3, 3, 3, 3]

Test Case 7:
Original: [-5, -10, 0, 5, 2]
Sorted: [-10, -5, 0, 2, 5]

Explanation : The merge_sort() function divides the list into smaller halves recursively until each part has one element, then the merge() function combines these parts by comparing elements to produce a sorted list in ascending order.

Complexity: Time Complexity = $O(n \log n)$, Space Complexity = $O(n)$.

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
 - o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.

- o Include docstrings explaining best, average, and worst-case complexities.
- o Test with various inputs.

```
[6]
✓ Os ▶ def binary_search(arr, target):
    """
    Performs Binary Search on a sorted list.

    Time Complexity:
    Best Case: O(1)      -> Target is found at the middle element.
    Average Case: O(log n) -> Search space is halved each step.
    Worst Case: O(log n) -> Element not present or found at the last step.

    Space Complexity:
    O(1) because the algorithm uses constant extra space.

    Parameters:
    arr (list): A sorted list of elements.
    target (int): The value to search for.

    Returns:
    int: Index of the target if found, otherwise -1.
    """

    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

```
[6] 0s
# ----- Test Cases -----
def test_binary_search():
    print("Running Test Cases...\n")

    test_cases = [
        ([1, 3, 5, 7, 9, 11], 7),
        ([2, 4, 6, 8, 10], 2),
        ([10, 20, 30, 40, 50], 35),
        ([], 5),
        ([1], 1),
        ([5, 10, 15, 20, 25], 25)
    ]

    for i, (arr, target) in enumerate(test_cases, 1):
        result = binary_search(arr, target)
        print(f"Test Case {i}:")
        print("Array:", arr)
        print("Target:", target)
        print("Result Index:", result)
        print("-" * 30)

# Run tests
test_binary_search()
```

Running Test Cases...

Test Case 1:
Array: [1, 3, 5, 7, 9, 11]
Target: 7
Result Index: 3

Test Case 2:
Array: [2, 4, 6, 8, 10]
Target: 2
Result Index: 0

Test Case 3:
Array: [10, 20, 30, 40, 50]
Target: 35
Result Index: -1

Test Case 4:
Array: []
Target: 5
Result Index: -1

Test Case 5:
Array: [1]
Target: 1
Result Index: 0

Test Case 6:
Array: [5, 10, 15, 20, 25]
Target: 25
Result Index: 4

Explanation : The `binary_search()` function repeatedly checks the middle element of the sorted list and compares it with the target, reducing the search space by half each time until the element is found or the list is exhausted.

Complexity: Best Case $O(1)$, Average & Worst Case $O(\log n)$.

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

```
[4] # Healthcare Appointment Management System
✓ Os #

# Sample Data
appointments = [
    {"id": "A104", "patient": "Ravi", "doctor": "Dr. Sharma", "time": "10:30", "fee": 500},
    {"id": "A101", "patient": "Priya", "doctor": "Dr. Mehta", "time": "09:00", "fee": 300},
    {"id": "A103", "patient": "Arjun", "doctor": "Dr. Rao", "time": "11:00", "fee": 700},
    {"id": "A102", "patient": "Sneha", "doctor": "Dr. Sharma", "time": "08:30", "fee": 300},
    {"id": "A105", "patient": "Kiran", "doctor": "Dr. Mehta", "time": "14:00", "fee": 450},
]

# -----
# 1. HASH MAP – Build index for O(1) search
# -----

def build_hash_map(appointments):
    hash_map = {}
    for appt in appointments:
        hash_map[appt["id"]] = appt    # ID is the key
    return hash_map

def search_by_id(hash_map, appt_id):
    if appt_id in hash_map:
        return hash_map[appt_id]    # O(1) lookup
    return "Appointment not found"

# -----
# 2. MERGE SORT – Sort by time or fee
# -----

def merge_sort(data, key):
    if len(data) <= 1:
        return data

    # Split into two halves
    mid = len(data) // 2
    left = merge_sort(data[:mid], key)
    right = merge_sort(data[mid:], key)

    return merge(left, right, key)
```

[4]
✓ Os



```
def merge(left, right, key):
    result = []
    i = j = 0

    # Compare and merge
    while i < len(left) and j < len(right):
        if left[i][key] <= right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# -----
# 3. DISPLAY HELPER
# -----

def display(appointments):
    print(f"{'ID':<6} {'Patient':<8} {'Doctor':<12} {'Time':<8} {'Fee'}")
    print("-" * 45)
    for a in appointments:
        print(f"{a['id']:<6} {a['patient']:<8} {a['doctor']:<12} {a['time']:<8} ₹{a['fee']}")
    print()

# -----
# MAIN - Run Everything
# -----

# Build hash map
hash_map = build_hash_map(appointments)

# Search
print(" Search by Appointment ID: A103")
print(search_by_id(hash_map, "A103"))
print()
```

```
# Search
print(" Search by Appointment ID: A103")
print(search_by_id(hash_map, "A103"))
print()

print("Search by Appointment ID: A999")
print(search_by_id(hash_map, "A999"))
print()

# Sort by Time
print("Sorted by Appointment Time:")
sorted_by_time = merge_sort(appointments, "time")
display(sorted_by_time)

# Sort by Fee
print("Sorted by Consultation Fee:")
sorted_by_fee = merge_sort(appointments, "fee")
display(sorted_by_fee)
```

```
... Search by Appointment ID: A103
{'id': 'A103', 'patient': 'Arjun', 'doctor': 'Dr. Rao', 'time': '11:00', 'fee': 700}

Search by Appointment ID: A999
Appointment not found

Sorted by Appointment Time:
ID      Patient  Doctor    Time    Fee
-----
A102    Sneha    Dr. Sharma 08:30   ₹300
A101    Priya    Dr. Mehta 09:00   ₹300
A104    Ravi     Dr. Sharma 10:30   ₹500
A103    Arjun    Dr. Rao   11:00   ₹700
A105    Kiran    Dr. Mehta 14:00   ₹450

Sorted by Consultation Fee:
ID      Patient  Doctor    Time    Fee
-----
A101    Priya    Dr. Mehta 09:00   ₹300
A102    Sneha    Dr. Sharma 08:30   ₹300
A105    Kiran    Dr. Mehta 14:00   ₹450
A104    Ravi     Dr. Sharma 10:30   ₹500
A103    Arjun    Dr. Rao   11:00   ₹700
```

Algorithm Selection For Searching → Hash Map (Dictionary)

Appointment IDs are unique → perfect for key-value lookup $O(1)$ average time — instant lookup regardless of data size

For Sorting → Merge Sort

Stable sort → preserves original order for equal values $O(n \log n)$ always — consistent performance Works well for both time and fee sorting

Justification : Hash Map is used for searching because it finds any appointment by ID instantly in $O(1)$ time, like a dictionary where you directly jump to the word instead of reading every page. Merge Sort is used for sorting because it always runs in $O(n \log n)$ time and is stable, meaning if two patients have the same fee (like ₹300), their original order is preserved — which matters in medical records.

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The

system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

```
[6]
✓ Os

# -----
# 2. BINARY SEARCH - Search by Ticket ID
# -----

def binary_search(sorted_data, ticket_id):
    low = 0
    high = len(sorted_data) - 1

    while low <= high:
        mid = (low + high) // 2
        mid_id = sorted_data[mid]["ticket_id"]

        if mid_id == ticket_id:
            return sorted_data[mid]      # ✓ Found
        elif mid_id < ticket_id:
            low = mid + 1                 # Search right half
        else:
            high = mid - 1                # Search left half

    return "Ticket not found"

# -----
# 3. DISPLAY HELPER
# -----

def display(bookings):
    print(f"{'Ticket ID':<12} {'Passenger':<10} {'Train':<8} {'Seat':<6} {'Date'}")
    print("-" * 50)
    for b in bookings:
        print(f"{b['ticket_id']:<12} {b['passenger']:<10} {b['train']:<8} {b['seat']:<6} {b['date']}")
    print()

# -----
# MAIN - Run Everything
# -----

# Step 1: Sort by ticket ID first (needed for binary search)
sorted_by_id = merge_sort(bookings, "ticket_id")
```

[4]
✓ Os



```
def merge(left, right, key):
    result = []
    i = j = 0

    # Compare and merge
    while i < len(left) and j < len(right):
        if left[i][key] <= right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# -----
# 3. DISPLAY HELPER
# -----

def display(appointments):
    print(f"{'ID':<6} {'Patient':<8} {'Doctor':<12} {'Time':<8} {'Fee'}")
    print("-" * 45)
    for a in appointments:
        print(f"{a['id']:<6} {a['patient']:<8} {a['doctor']:<12} {a['time']:<8} ₹{a['fee']}")
    print()

# -----
# MAIN - Run Everything
# -----

# Build hash map
hash_map = build_hash_map(appointments)

# Search
print(" Search by Appointment ID: A103")
print(search_by_id(hash_map, "A103"))
print()
```

```
[6] 0s print("Search Ticket ID: 1999")
result = binary_search(sorted_by_id, 1999)
print(result)
print()

# Sort by Travel Date
print("Sorted by Travel Date:")
sorted_by_date = merge_sort(bookings, "date")
display(sorted_by_date)

# Sort by Seat Number
print("Sorted by Seat Number:")
sorted_by_seat = merge_sort(bookings, "seat")
display(sorted_by_seat)
```

*** Search Ticket ID: 1003
{'ticket_id': 1003, 'passenger': 'Ravi', 'train': '12345', 'seat': 24, 'date': '2026-03-25'}

Search Ticket ID: 1999
Ticket not found

Sorted by Travel Date:

Ticket ID	Passenger	Train	Seat	Date
1001	Priya	67890	10	2026-03-21
1002	Sneha	12345	10	2026-03-21
1003	Ravi	12345	24	2026-03-25
1004	Kiran	67890	18	2026-03-26
1005	Arjun	11223	5	2026-03-28

Sorted by Seat Number:

Ticket ID	Passenger	Train	Seat	Date
1005	Arjun	11223	5	2026-03-28
1001	Priya	67890	10	2026-03-21
1002	Sneha	12345	10	2026-03-21
1004	Kiran	67890	18	2026-03-26
1003	Ravi	12345	24	2026-03-25

Algorithm Selection For Searching → Binary Search

Ticket IDs are numeric & sorted → perfect for binary search $O(\log n)$ time — cuts search space in half each step

For Sorting → Tim Sort (Python's built-in) / Merge Sort

Stable sort → preserves order for equal seat numbers $O(n \log n)$ always — consistent performance

Justification : Binary Search is used for searching because ticket IDs are numeric and sorted, so it finds any ticket in $O(\log n)$ time by cutting the list in half each step — like finding a page in a book by opening the middle first. Merge Sort is used for sorting because it always runs in $O(n \log n)$ time and is stable, meaning if two passengers have the same seat number or travel date (like Priya & Sneha both on 2026-03-21), their original order is safely preserved.

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

```

[7] # -----
✓ Os # Smart Hostel Room Allocation System
# -----

# Sample Data
allocations = [
    {"student_id": "S104", "name": "Ravi", "room": 204, "floor": 2, "date": "2026-01-15"},
    {"student_id": "S101", "name": "Priya", "room": 101, "floor": 1, "date": "2026-01-10"},
    {"student_id": "S103", "name": "Arjun", "room": 301, "floor": 3, "date": "2026-01-20"},
    {"student_id": "S102", "name": "Sneha", "room": 101, "floor": 1, "date": "2026-01-12"},
    {"student_id": "S105", "name": "Kiran", "room": 202, "floor": 2, "date": "2026-01-18"},
]

# -----

# 1. HASH MAP – Build index for O(1) search
# -----

def build_hash_map(allocations):
    hash_map = {}
    for record in allocations:
        hash_map[record["student_id"]] = record # Student ID as key
    return hash_map

def search_by_id(hash_map, student_id):
    if student_id in hash_map:
        return hash_map[student_id] # O(1) direct lookup
    return "Student record not found"

# -----

# 2. MERGE SORT – Sort by room or date
# -----

def merge_sort(data, key):
    if len(data) <= 1:
        return data

    # Split into two halves
    mid = len(data) // 2
    left = merge_sort(data[:mid], key)
    right = merge_sort(data[mid:], key)

    return merge(left, right, key)

def merge(left, right, key):
    result = []
    i = j = 0

    # Compare and merge both halves
    while i < len(left) and j < len(right):
        if left[i][key] <= right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])

```

```

[?] 3. DISPLAY HELPER
✓ Os #
# -----

def display(allocations):
    print(f"{'Student ID':<12} {'Name':<8} {'Room':<6} {'Floor':<7} {'Date'}")
    print("-" * 50)
    for a in allocations:
        print(f"{a['student_id']:<12} {a['name']:<8} {a['room']:<6} {a['floor']:<7} {a['date']}")
    print()

# -----
# MAIN - Run Everything
# -----

# Build Hash Map
hash_map = build_hash_map(allocations)

# Search
print("Search Student ID: S103")
print(search_by_id(hash_map, "S103"))
print()

print("Search Student ID: S999")
print(search_by_id(hash_map, "S999"))
print()

# Sort by Room Number
print("Sorted by Room Number:")
sorted_by_room = merge_sort(allocations, "room")
display(sorted_by_room)

# Sort by Allocation Date
print("Sorted by Allocation Date:")
sorted_by_date = merge_sort(allocations, "date")
display(sorted_by_date)

```

... Search Student ID: S103
 {'student_id': 'S103', 'name': 'Arjun', 'room': 301, 'floor': 3, 'date': '2026-01-20'}

Search Student ID: S999
 Student record not found

Sorted by Room Number:

Student ID	Name	Room	Floor	Date
S101	Priya	101	1	2026-01-10
S102	Sneha	101	1	2026-01-12
S105	Kiran	202	2	2026-01-18
S104	Ravi	204	2	2026-01-15
S103	Arjun	301	3	2026-01-20

Sorted by Allocation Date:

Student ID	Name	Room	Floor	Date
S101	Priya	101	1	2026-01-10
S102	Sneha	101	1	2026-01-12
S104	Ravi	204	2	2026-01-15
S105	Kiran	202	2	2026-01-18
S103	Arjun	301	3	2026-01-20

Algorithm Selection For Searching → Hash Map (Dictionary)

Student IDs are unique → perfect for direct key-value lookup $O(1)$ average time — instant access regardless of data size

For Sorting → Merge Sort

Stable sort → preserves order for equal room numbers or dates $O(n \log n)$ always — consistent and reliable performance

Justification : Hash Map is used for searching because student IDs are unique, so it finds any record in $O(1)$ time instantly — like a locker system where each student has a unique key to their locker. Merge Sort is used for sorting because it always runs in $O(n \log n)$ time and is stable, meaning if two students share the same room number (like Priya & Sneha both in Room 101), their original allocation order is safely preserved — which is critical in hostel management.

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.

[8]
✓ Os

```
# -----  
# Online Movie Streaming Platform  
# -----  
  
# Sample Data  
movies = [  
    {"movie_id": "M104", "title": "Interstellar", "genre": "Sci-Fi", "rating": 8.6, "year": 2014},  
    {"movie_id": "M101", "title": "The Dark Knight", "genre": "Action", "rating": 9.0, "year": 2008},  
    {"movie_id": "M103", "title": "Inception", "genre": "Sci-Fi", "rating": 8.8, "year": 2010},  
    {"movie_id": "M102", "title": "Avengers", "genre": "Action", "rating": 8.4, "year": 2012},  
    {"movie_id": "M105", "title": "Pushpa", "genre": "Drama", "rating": 7.6, "year": 2021},  
]
```

1. HASH MAP – Build index for O(1) search

```
def build_hash_map(movies):  
    hash_map = {}  
    for movie in movies:  
        hash_map[movie["movie_id"]] = movie    # Movie ID as key  
    return hash_map  
  
def search_by_id(hash_map, movie_id):  
    if movie_id in hash_map:  
        return hash_map[movie_id]              # O(1) direct lookup  
    return "Movie not found"
```

2. MERGE SORT – Sort by rating or year

```
def merge_sort(data, key, reverse=False):  
    if len(data) <= 1:  
        return data  
  
    # Split into two halves  
    mid = len(data) // 2  
    left = merge_sort(data[:mid], key, reverse)  
    right = merge_sort(data[mid:], key, reverse)  
  
    return merge(left, right, key, reverse)  
  
def merge(left, right, key, reverse):  
    result = []  
    i = j = 0  
  
    # Compare and merge both halves  
    while i < len(left) and j < len(right):  
        # reverse=True → descending (highest rating first)  
        # reverse=False → ascending (oldest year first)  
        if reverse:  
            condition = left[i][key] >= right[j][key]  
        else:  
            condition = left[i][key] <= right[j][key]  
  
        if condition:  
            result.append(left[i])  
            i += 1
```



```

        if condition:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# -----
# 3. DISPLAY HELPER
# -----

def display(movies):
    print(f"{'Movie ID':<10} {'Title':<20} {'Genre':<10} {'Rating':<8} {'Year'}")
    print("-" * 58)
    for m in movies:
        print(f"{'movie_id':<10} {'title':<20} {'genre':<10} {'rating':<8} {'year'}")
    print()

# -----
# MAIN - Run Everything
# -----

# Build Hash Map
hash_map = build_hash_map(movies)

# Search
print(" Search Movie ID: M103")
print(search_by_id(hash_map, "M103"))
print()

print(" Search Movie ID: M999")
print(search_by_id(hash_map, "M999"))
print()

# Sort by Rating (Highest First)
print("Sorted by Rating (High + Low):")
sorted_by_rating = merge_sort(movies, "rating", reverse=True)
display(sorted_by_rating)

# Sort by Release Year (Oldest First)
print("Sorted by Release Year (Old + New):")
sorted_by_year = merge_sort(movies, "year", reverse=False)
display(sorted_by_year)

```



```

[9] #
✓ Os # Smart Agriculture Crop Monitoring System
#

# Sample Data
crops = [
    {"crop_id": "C104", "name": "Wheat", "moisture": 65.0, "temperature": 28.5, "yield": 4200},
    {"crop_id": "C101", "name": "Rice", "moisture": 80.0, "temperature": 32.0, "yield": 5500},
    {"crop_id": "C103", "name": "Maize", "moisture": 55.0, "temperature": 30.0, "yield": 3800},
    {"crop_id": "C102", "name": "Sugarcane", "moisture": 75.0, "temperature": 35.0, "yield": 7000},
    {"crop_id": "C105", "name": "Cotton", "moisture": 55.0, "temperature": 33.0, "yield": 2900},
]

#
# 1. HASH MAP - Build index for O(1) search
#

def build_hash_map(crops):
    hash_map = {}
    for crop in crops:
        hash_map[crop["crop_id"]] = crop    # Crop ID as key
    return hash_map

def search_by_id(hash_map, crop_id):
    if crop_id in hash_map:
        return hash_map[crop_id]            # O(1) direct lookup
    return "Crop record not found"

#
# 2. MERGE SORT - Sort by moisture or yield
#

def merge_sort(data, key, reverse=False):
    if len(data) <= 1:
        return data

    # Split into two halves
    mid = len(data) // 2
    left = merge_sort(data[:mid], key, reverse)
    right = merge_sort(data[mid:], key, reverse)

    return merge(left, right, key, reverse)

def merge(left, right, key, reverse):
    result = []
    i = j = 0

    # Compare and merge both halves
    while i < len(left) and j < len(right):
        # reverse=True -> descending (highest yield first)
        # reverse=False -> ascending (lowest moisture first)
        if reverse:
            condition = left[i][key] >= right[j][key]
        else:
            condition = left[i][key] <= right[j][key]

        if condition:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Append remaining elements
    result.extend(left[i:])
    result.extend(right[j:])

    return result

```

```

[9]
✓ Os
else:
    result.append(right[j])
    j += 1

# Add remaining elements
result.extend(left[i:])
result.extend(right[j:])
return result

# -----
# 3. DISPLAY HELPER
# -----

def display(crops):
    print(f"{'Crop ID':<10} {'Name':<12} {'Moisture%':<12} {'Temp(°C)':<10} {'Yield(kg)'}")
    print("-" * 58)
    for c in crops:
        print(f"{c['crop_id']:<10} {c['name']:<12} {c['moisture']:<12} {c['temperature']:<10} {c['yield']}")
    print()

# -----
# MAIN - Run Everything
# -----

# Build Hash Map
hash_map = build_hash_map(crops)

# Search
print(" Search Crop ID: C102")
print(search_by_id(hash_map, "C102"))
print()

print("Search Crop ID: C999")
print(search_by_id(hash_map, "C999"))
print()

# Sort by Moisture Level (Low → High)
print(" Sorted by Moisture Level (Low → High):")
sorted_by_moisture = merge_sort(crops, "moisture", reverse=False)
display(sorted_by_moisture)

# Sort by Yield Estimate (High → Low)
print(" & Sorted by Yield Estimate (High → Low):")
sorted_by_yield = merge_sort(crops, "yield", reverse=True)
display(sorted_by_yield)

```

```

Search Crop ID: C102
... {'crop_id': 'C102', 'name': 'Sugarcane', 'moisture': 75.0, 'temperature': 35.0, 'yield': 7000}

Search Crop ID: C999
Crop record not found

Sorted by Moisture Level (Low → High):
Crop ID   Name      Moisture%  Temp(°C)  Yield(kg)
-----
C103      Maize      55.0       30.0      3800
C105      Cotton     55.0       33.0      2900
C104      Wheat      65.0       28.5      4200
C102      Sugarcane  75.0       35.0      7000
C101      Rice       80.0       32.0      5500

🔪 Sorted by Yield Estimate (High → Low):
Crop ID   Name      Moisture%  Temp(°C)  Yield(kg)
-----
C102      Sugarcane  75.0       35.0      7000
C101      Rice       80.0       32.0      5500
C104      Wheat      65.0       28.5      4200
C103      Maize      55.0       30.0      3800
C105      Cotton     55.0       33.0      2900

```

Algorithm Selection For Searching → Hash Map (Dictionary)

Crop IDs are unique → perfect for direct key-value lookup $O(1)$ average time — instant access regardless of data size

For Sorting → Merge Sort

Stable sort → preserves order for equal moisture levels or yield estimates $O(n \log n)$ always — consistent and reliable performance

Justification : Hash Map is used for searching because crop IDs are unique, so it finds any crop record in $O(1)$ time instantly — like a farmer's register where each crop has a unique tag for direct access without checking every record. Merge Sort is used for sorting because it always runs in $O(n \log n)$ time and is stable, meaning if two crops have the same moisture level (like Maize & Cotton both at 55.0%), their original order is safely preserved — which is critical when farmers need accurate priority-based irrigation scheduling.

Search Crop ID: C999
Crop record not found

Sorted by Moisture Level (Low → High):

Crop ID	Name	Moisture%	Temp(°C)	Yield(kg)
C103	Maize	55.0	30.0	3800
C105	Cotton	55.0	33.0	2900
C104	Wheat	65.0	28.5	4200
C102	Sugarcane	75.0	35.0	7000
C101	Rice	80.0	32.0	5500

👉 Sorted by Yield Estimate (High → Low):

Crop ID	Name	Moisture%	Temp(°C)	Yield(kg)
C102	Sugarcane	75.0	35.0	7000
C101	Rice	80.0	32.0	5500
C104	Wheat	65.0	28.5	4200
C103	Maize	55.0	30.0	3800
C105	Cotton	55.0	33.0	2900

Algorithm Selection For Searching → Hash Map (Dictionary)

Crop IDs are unique → perfect for direct key-value lookup $O(1)$ average time — instant access regardless of data size

For Sorting → Merge Sort

Stable sort → preserves order for equal moisture levels or yield estimates $O(n \log n)$ always — consistent and reliable performance

Justification : Hash Map is used for searching because crop IDs are unique, so it finds any crop record in $O(1)$ time instantly — like a farmer's register where each crop has a unique tag for direct access without checking every record. Merge Sort is used for sorting because it always runs in $O(n \log n)$ time and is stable, meaning if two crops have the same moisture level (like Maize & Cotton both at 55.0%), their original order is safely preserved — which is critical when farmers need accurate priority-based irrigation scheduling.

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

[10]
✓ Os

```
# -----  
# Airport Flight Management System  
# -----  
  
# Sample Data  
flights = [  
    {"flight_id": "F104", "airline": "Indigo", "departure": "10:30", "arrival": "13:00", "status": "On Time"},  
    {"flight_id": "F101", "airline": "Air India", "departure": "08:00", "arrival": "11:30", "status": "Delayed"},  
    {"flight_id": "F103", "airline": "SpiceJet", "departure": "09:45", "arrival": "12:15", "status": "On Time"},  
    {"flight_id": "F102", "airline": "Vistara", "departure": "08:00", "arrival": "10:30", "status": "Cancelled"},  
    {"flight_id": "F105", "airline": "GoFirst", "departure": "14:00", "arrival": "17:30", "status": "On Time"},  
]  
  
# -----  
# 1. HASH MAP - Build index for O(1) search  
# -----  
  
def build_hash_map(flights):  
    hash_map = {}  
    for flight in flights:  
        hash_map[flight["flight_id"]] = flight # Flight ID as key  
    return hash_map  
  
def search_by_id(hash_map, flight_id):  
    if flight_id in hash_map:  
        return hash_map[flight_id] # O(1) direct lookup  
    return "Flight not found"  
  
# -----  
# 2. MERGE SORT - Sort by departure or arrival  
# -----  
  
def merge_sort(data, key, reverse=False):  
    if len(data) <= 1:  
        return data  
  
    # Split into two halves  
    mid = len(data) // 2  
    left = merge_sort(data[:mid], key, reverse)  
    right = merge_sort(data[mid:], key, reverse)  
  
    return merge(left, right, key, reverse)  
  
def merge(left, right, key, reverse):  
    result = []  
    i = j = 0  
  
    # Compare and merge both halves  
    while i < len(left) and j < len(right):  
        # reverse=True -> descending (latest time first)  
        # reverse=False -> ascending (earliest time first)  
        if reverse:  
            condition = left[i][key] >= right[j][key]  
        else:  
            condition = left[i][key] <= right[j][key]  
  
        if condition:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
    result.extend(left[i:] if i < len(left) else right[j:])  
    return result
```

```

        if condition:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# -----
# 3. DISPLAY HELPER
# -----

def display(flights):
    print(f"{'Flight ID':<11} {'Airline':<12} {'Departure':<11} {'Arrival':<10} {'Status'}")
    print("-" * 58)
    for f in flights:
        print(f"{'flight_id':<11} {'airline':<12} {'departure':<11} {'arrival':<10} {'status'}")
    print()

# -----
# MAIN - Run Everything
# -----

# Build Hash Map
hash_map = build_hash_map(flights)

# Search
print(" Search Flight ID: F103")
print(search_by_id(hash_map, "F103"))
print()

print(" Search Flight ID: F999")
print(search_by_id(hash_map, "F999"))
print()

# Sort by Departure Time (Earliest First)
print(" Sorted by Departure Time (Early + Late):")
sorted_by_departure = merge_sort(flights, "departure", reverse=False)
display(sorted_by_departure)

# Sort by Arrival Time (Earliest First)
print("Sorted by Arrival Time (Early + Late):")
sorted_by_arrival = merge_sort(flights, "arrival", reverse=False)
display(sorted_by_arrival)

```

display(sorted_by_arrival)

Search Flight ID: F103
{'flight_id': 'F103', 'airline': 'SpiceJet', 'departure': '09:45', 'arrival': '12:15', 'status': 'On Time'}

Search Flight ID: F999
Flight not found

Sorted by Departure Time (Early → Late):

Flight ID	Airline	Departure	Arrival	Status
F101	Air India	08:00	11:30	Delayed
F102	Vistara	08:00	10:30	Cancelled
F103	SpiceJet	09:45	12:15	On Time
F104	IndiGo	10:30	13:00	On Time
F105	GoFirst	14:00	17:30	On Time

Sorted by Arrival Time (Early → Late):

Flight ID	Airline	Departure	Arrival	Status
F102	Vistara	08:00	10:30	Cancelled
F101	Air India	08:00	11:30	Delayed
F103	SpiceJet	09:45	12:15	On Time
F104	IndiGo	10:30	13:00	On Time
F105	GoFirst	14:00	17:30	On Time

Algorithm Selection For Searching → Hash Map (Dictionary)

Flight IDs are unique → perfect for direct key-value lookup $O(1)$ average time — instant access regardless of data size

For Sorting → Merge Sort

Stable sort → preserves order for equal departure or arrival times $O(n \log n)$ always — consistent and reliable performance

Justification : Hash Map is used for searching because flight IDs are unique, so it finds any flight in $O(1)$ time instantly — like an airport display board where entering a flight number directly shows its gate and status without scanning every flight. Merge Sort is used for sorting because it always runs in $O(n \log n)$ time and is stable, meaning if two flights have the same departure time (like Air India & Vistara both at 08:00), their original order is safely preserved — which is critical in airports to avoid scheduling conflicts and runway mismanagement.